

Brevo Email Integration Guide

GTP 2026 Bootcamp – Project Management Tool



```
$ whoami
opiyo-don-paul (unchanted)

$ cat sprint-backlog.txt | grep email
[ ] transactional email – Brevo, not wired yet

$ ./setup-brevo.sh --beginner --until-it-works
running 9 steps... see next page
```

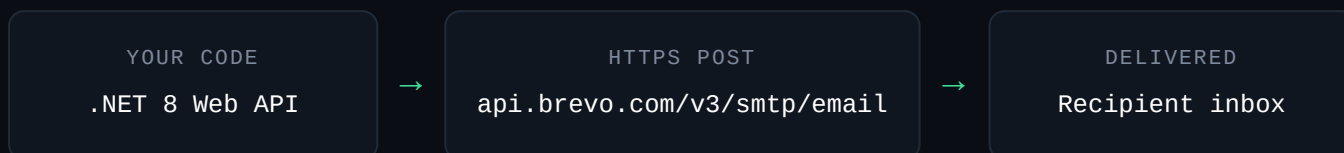
What this guide gets you to

By the end you'll have a Brevo account with a verified sender, an API key, a tested send, and working **.NET 8** code in your Project Management Tool backend that sends real emails — plus a prompt you can hand to an AI to finish the wiring for you.

The 9 steps

- | | |
|--|---|
|  01 · Create a Brevo account |  02 · Verify your account email |
|  03 · Add & verify a sender |  04 · Authenticate a domain (optional) |
|  05 · Generate your API key |  06 · Test with a raw curl call |
|  07 · Wire it into .NET 8 |  08 · Know the common failure modes |
|  09 · Hand the rest to an AI | |

How mail actually moves



The Brevo *dashboard* (senders, domains, API keys, delivery logs) sits beside this flow — it configures the pipe, it doesn't carry the emails.

WHY BREVO

Free tier gives you 300 transactional emails/day, a real REST API, and per-message delivery logs — enough for a bootcamp capstone and most early-stage products. No credit card needed to start.

1. Go to **brevo.com** and click **Sign up free**.
2. Enter your email, a password, first/last name — or continue with Google.
3. Brevo emails you a confirmation link immediately. Leave the tab open.

WHICH EMAIL TO SIGN UP WITH

Use an inbox you actually check — your account email doubles as your first sender candidate and where security/billing notices land. A personal Gmail is fine to start; you can move to a branded domain later without losing anything (Step 4).

What onboarding will ask

After you confirm your email, Brevo runs a short wizard: company/site name, what you're sending (pick anything close to "transactional / product notifications"), and team size. None of it is binding — skip anything optional and land on the dashboard.

IF YOU ALREADY HAVE AN ACCOUNT

Skip to Step 3 — your screenshot shows an account that already exists, with senders already added. Steps 1–2 are here for completeness / for teammates repeating this setup.

1. Open the "Confirm your Brevo account" email and click the verification link.
2. You're dropped back into the dashboard, now fully activated (some features stay locked behind this until it's done — including sending).
3. Didn't get it? Check spam, then use "Resend confirmation email" on the banner Brevo shows while unverified.

TWO DIFFERENT VERIFICATIONS — DON'T MIX THESE UP

Account verification (this step) proves you own the login email and unlocks the dashboard. **Sender** verification (Step 3) proves you own whatever address you want emails to be sent *from* — a separate, per-sender check, even if it's the same address.

Where you land

The main dashboard, left nav split into *Personal settings*, *Organization settings* (this is where Senders, domains, IPs and SMTP & API live), *CRM*, *Marketing*, *Conversations* and *Automations*. Everything in this guide lives under **Organization settings**.



Add & verify a sender

Settings → Senders, domains, IPs → Add sender

- 1. Click the gear icon (top right) → **Senders, domains, IPs**.
- 2. Click **Add sender** → enter a display **name** (what recipients see, e.g. "Project Management Tool") and the **email** you're sending from.
- 3. Brevo emails that inbox a 6-digit code (subject: "[Brevo] Activate Your New Sender").
- 4. Paste the code into the verification prompt → sender flips to **Verified**.

Senders, Domains & Dedicated IPs

Add sender

Senders

Domains

Dedicated IPs

Sending cadence

Senders

A sender is the name and email address that help recipients recognize your brand and feel confident opening your emails. To use a sender, it must be verified. For better email deliverability, your sender's domain should also be authenticated.

One or several of your senders are not compliant with [Google, Yahoo, and Microsoft's new requirements for senders](#). For each sender, check their DKIM signature and DMARC status and take the recommended action to be able to use it for email sending.

Search sender by name or email

Griot <info.donartkins.ke@gmail.com>

Verified

Edit

IP address

DKIM signature

DMARC

Shared IP

Default

Freemail domain is not recommended

Lyncxs Industries <info.donartkins.ke@gmail.com>

Verified

Edit

IP address

DKIM signature

DMARC

Shared IP

Default

Freemail domain is not recommended

Your account – two verified senders, both flagged for domain authentication.

READING YOUR OWN SCREENSHOT

Verified (green dot) just means you proved you own that inbox — sending already works. The **DKIM: Default** and **DMARC: Freemail domain is not recommended** warnings appear because the sender is a Gmail address: you don't control gmail.com's DNS, so Brevo can't cryptographically sign mail as that domain. It still sends; it just carries less trust, so more of it lands in Spam/Promotions than a real registered domain would. Step 4 is the actual fix — optional for a bootcamp demo, worth doing before real users.

BREVO SETUP GUIDE

PAGE 5

Authenticate a domain

optional, but this is the real deliverability fix

Only possible if you **own a domain** (a `vercel.app` or `github.io` subdomain doesn't count — you don't control its DNS). If you don't own one yet, skip to Step 5 and come back later; nothing here blocks the rest of the guide.

1. Senders, domains, IPs → **Domains** tab → **Add a domain**.
2. Brevo gives you 2–3 DNS records to publish: a verification **TXT** record and a **DKIM** record (Brevo will also show a **DMARC** record if your account needs one for the new Google/Yahoo/Microsoft sender rules).
3. Add those records at your domain registrar's DNS settings (Namecheap, Cloudflare, etc).
4. Back in Brevo, click **Verify**. DNS propagation can take a few minutes to a few hours.
5. Once the domain shows **Authenticated**, change your sender's email to an address on that domain (e.g. `notifications@yourdomain.com`) — no code change needed later, since your app should only ever read the sender address from config (Step 7).

SPF, BRIEFLY

You generally don't need to add an SPF record for Brevo's own sending domain — Brevo controls the technical "envelope from" and handles that alignment for you. Only the verification TXT + DKIM (+ DMARC when prompted) are required on your side.

COMMON TRAP

Hosting your frontend on a shared platform (Vercel/Netlify/GitHub Pages default subdomain) does **not** give you a domain you can authenticate — you'd need to buy a real domain and point it at that host, then authenticate *that* domain in Brevo.

1. Gear icon → **SMTP & API** → the **API Keys** tab.
2. Click **Generate a new API key**.
3. Name it something you'll recognize later — e.g. `pm-tool-dev` or `pm-tool-production`. One key per environment is good practice.
4. Click **Generate** and **copy it immediately** — Brevo shows the full key exactly once. Lose it and you generate a new one, not recover the old.

NEVER COMMIT THIS

Put it in `appsettings.Local.json` (git-ignored) or user-secrets / environment variables — never in `appsettings.json` that ships to your repo. A leaked key lets anyone send email as you and burns your daily quota.

dotnet CLI — user-secrets (recommended for dev)

bash

```
# run from your API project folder
dotnet user-secrets init
dotnet user-secrets set "Brevo:ApiKey" "xkeysib-your-real-key-here"
dotnet user-secrets set "Brevo:FromEmail" "your-verified-sender@example.com"
dotnet user-secrets set "Brevo:FromName" "Project Management Tool"
```

Keys don't expire on their own — you deactivate/regenerate manually from this same page if one leaks.

Test it before you write app code

one curl call — matches the Postman testing your bootcamp already uses

Confirm the account, sender and key actually work end-to-end before wiring anything into your API. If this fails, it's a Brevo/dashboard problem, not a bug in your code.

terminal / Postman raw body

bash

```
curl -i -X POST https://api.brevo.com/v3/smtp/email \
-H "accept: application/json" \
-H "api-key: YOUR_API_KEY_HERE" \
-H "content-type: application/json" \
-d '{
  "sender": {"name":"Project Management Tool","email":"YOUR_VERIFIED_SENDER"},
  "to": [{"email":"YOUR_OWN_EMAIL","name":"Test"}],
  "subject": "Brevo test email",
  "htmlContent": "<h1>It works</h1><p>Sent from the PM Tool capstone.</p>"
}'
```

- `sender.email` must exactly match a **Verified** sender from Step 3.
- A `201` with a `messageId` means it's queued — check the inbox (and spam).
- Track every send under **Transactional** → **Email Activity** in the dashboard, with per-message delivery status.

IEmailService → BrevoEmailService, matching your bootcamp's ASP.NET Core stack

appsettings.json – shape only, real values go in user-secrets

json

```
{
  "Brevo": {
    "ApiKey": "",
    "FromEmail": "",
    "FromName": "Project Management Tool"
  }
}
```

Email/BrevoOptions.cs

C#

```
namespace Pmt.Infrastructure.Email;

public class BrevoOptions
{
    public const string SectionName = "Brevo";
    public string ApiKey { get; set; } = "";
    public string FromEmail { get; set; } = "";
    public string FromName { get; set; } = "";
}
```

Application/Email/IEmailService.cs

C#

```
namespace Pmt.Application.Email;

public interface IEmailService
{
    Task<bool> SendAsync(string toEmail, string toName, string subject, string htmlBody);
}
```

Program.cs – DI wiring

C#

```
builder.Services.Configure<BrevoOptions>
(builder.Configuration.GetSection(BrevoOptions.SectionName));
builder.Services.AddHttpClient<IEmailService, BrevoEmailService>(client =>
{
    client.BaseAddress = new Uri("https://api.brevo.com/v3/");
    client.DefaultRequestHeaders.Add("api-key", builder.Configuration["Brevo:ApiKey"]);
    client.DefaultRequestHeaders.Add("accept", "application/json");
});
```

The service that talks to Brevo

Infrastructure/Email/BrevoEmailService.cs

C#

```

using System.Net.Http.Json;
using Microsoft.Extensions.Options;
using Pmt.Application.Email;

namespace Pmt.Infrastructure.Email;

public class BrevoEmailService : IEmailService
{
    private readonly HttpClient _http;
    private readonly BrevoOptions _options;
    private readonly ILogger<BrevoEmailService> _logger;

    public BrevoEmailService(HttpClient http, IOptions<BrevoOptions> options,
        ILogger<BrevoEmailService> logger)
    {
        _http = http; _options = options.Value; _logger = logger; }

    public async Task<bool> SendAsync(string toEmail, string toName, string subject, string
        htmlBody)
    {
        var payload = new
        {
            sender = new { name = _options.FromName, email = _options.FromEmail },
            to = new[] { new { email = toEmail, name = toName } },
            subject, htmlContent = htmlBody
        };
        try
        {
            var res = await _http.PostAsJsonAsync("smtp/email", payload);
            if (!res.IsSuccessStatusCode)
            {
                _logger.LogWarning("Brevo rejected email to {Email}: {Status}", toEmail,
                    res.StatusCode);
                return false;
            }
            return true;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Brevo send failed for {Email}", toEmail);
            return false; // a broken inbox should never break the API response
        }
    }
}

```

A template, and calling it from a real endpoint

Infrastructure/Email/EmailTemplates.cs – inline CSS only, email clients ignore stylesheets

C#

```
public static class EmailTemplates
{
    public static string TaskAssigned(string userName, string taskTitle, string projectName,
    string appUrl) =>
        $"""
            <body style="font-family:Arial,sans-serif;background:#f4f5f7;padding:24px;">
                <div style="max-width:480px;margin:auto;background:#fff;border-
radius:8px;padding:32px;">
                    <h2 style="color:#1f2937;">New task assigned</h2>
                    <p>Hi {userName},</p>
                    <p>You've been assigned <strong>{taskTitle}</strong> in <strong>{projectName}</strong>.</p>
                    <a href="{appUrl}" style="display:inline-block;background:#4f46e5;color:#fff;
padding:12px 20px;border-radius:6px;text-decoration:none;">Open task</a>
                </div>
            </body>
            """;
}
```

TasksController.cs – best-effort send, after the commit

C#

```
await _db.SaveChangesAsync(); // the assignment is real regardless of email

_ = _emailService.SendAsync(
    assignee.Email, assignee.FullName, "New task assigned",
    EmailTemplates.TaskAssigned(assignee.FullName, task.Title, project.Name, appUrl:
"https://your-app.vercel.app"));

return Ok(task);
```

WHY BEST-EFFORT

`SendAsync` already swallows failures and returns `false` — it never throws. That means a Brevo outage can't roll back a task assignment or fail a 200 response. If a specific flow genuinely needs the caller to know delivery failed (e.g. a login OTP), check the return value and surface it there instead — don't make every caller do it.



Common failure modes

check this before you assume it's your code

SYMPTOM	LIKELY CAUSE	FIX
400 "sender is not valid"	The <code>sender.email</code> you sent doesn't exactly match a Verified sender	Re-check Senders, domains, IPs — copy the address exactly
401 Unauthorized	Missing/wrong <code>api-key</code> header, or key was regenerated	Generate a fresh key (Step 5) and update user-secrets/env vars
Sends, but lands in Spam/Promotions	Freemail sender, no domain auth, or subject reads as marketing	Authenticate a domain (Step 4); keep transactional subjects plain
DKIM shows "Default" with a warning	Expected for any Gmail/Yahoo/Outlook sender — you don't own that domain's DNS	Fine for dev/demo; authenticate your own domain before real users
Hit the 300/day cap	Free plan daily limit, resets every 24h	Wait for reset, or add your own send-rate guardrails per user/IP
Nothing arrives, no error	Recipient server silently rejected, or typo'd address	Transactional → Email Activity — shows per-message status

ONE ACCOUNT-WIDE BUDGET

Your own rate limits (per-user, per-IP) stop abuse of a single endpoint — they don't stop you from collectively burning the 300/day cap across every feature that sends mail (OTP, invites, notifications). Watch it once more than one feature emails people.

Hand the rest to an AI

copy this into Claude / Claude Code / Cursor – attach this PDF if the tool takes files

Everything above is enough for an AI coding assistant to finish wiring this into your exact codebase. Fill the brackets, paste the block.

prompt.txt

copy-paste

I'm building a Project Management Tool (ASP.NET Core Web API, .NET 8) for my GTP 2026 bootcamp capstone. I have a working Brevo transactional-email account already: a verified sender and an API key, stored as Brevo:ApiKey / Brevo:FromEmail / Brevo:FromName.

Implement email sending following this pattern (full spec in the attached PDF, Step 7):

1. IEmailService with one method: SendAsync(toEmail, toName, subject, htmlBody) -> Task<bool>.
2. BrevoEmailService: a typed HttpClient that POSTs to `https://api.brevo.com/v3/smtp/email` with the api-key header, matching Brevo's request shape exactly.
3. Bind Brevo:* from configuration via `IOptions<BrevoOptions>`.
4. Never let a failed send throw or block the caller -- log it, return false. The API response must succeed even if Brevo is down.
5. Register with `AddHttpClient<IEmailService, BrevoEmailService>()` in Program.cs.
6. Add two templates as HTML-returning methods with inline CSS: "task assigned" and "welcome / account created".
7. Call SendAsync as a best-effort, fire-and-forget send from [YOUR TASK-ASSIGNMENT ENDPOINT] and [YOUR REGISTER ENDPOINT] -- after the DB commit, never before.

Ask me for my exact namespace, controller and DbContext names before generating code so it matches my project instead of guessing.

BEFORE YOU PASTE IT

Swap the two bracketed endpoint names for your real controller actions. If your AI tool doesn't accept file attachments, paste Step 7's code blocks from this PDF above the prompt instead of saying "attached."

Cheat sheet

Dashboard paths

- Add/verify sender — Settings → Senders, domains, IPs → Senders
- Authenticate domain — ...IPs → Domains → Add a domain
- API key — Settings → SMTP & API → API Keys
- Delivery logs — Transactional → Email Activity
- Daily quota used — Usage and plan (top bar)

Endpoint

Base

REST

```
POST https://api.brevo.com/v3/smtp/email
Header: api-key: <your key>
Header: content-type: application/json
```

Minimal request body

JSON

body

```
{
  "sender": {"name": "", "email": ""},
  "to": [{"email": "", "name": ""}],
  "subject": "",
  "htmlContent": "<p>...</p>"
}
```

Limits (free plan)

- 300 emails / day, resets every 24h
- API key shown once — regenerate if lost
- 50 test-template sends / day

SHIP ORDER

Sender verified → curl test passes → wire into code → one real end-to-end send from your app → then worry about domain authentication and templates. Don't design the perfect email template before you've confirmed a single one arrives.